13.2 会话路由源码分析

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~250,000 tokens，输出 ~18,000 tokens（本章合计）

一条消息从通道到达 Gateway 时，系统需要回答两个问题：**交给哪个 Agent?** 和 **放入哪个会话?** 这就是会话路由的核心职责。本节逐文件分析路由系统的源码实现。

13.2.1 路由绑定（Bindings）

路由绑定（Binding）是用户在配置文件中定义的规则，用于将特定的通道/账户/对等方映射到特定的 Agent。 `src/routing/bindings.ts` 提供了绑定查询的基础设施。

```
// src/routing/bindings.ts
import type { OpenClawConfig } from "../config/config.js";
import type { AgentBinding } from "../config/types.agents.js";

export function listBindings(cfg: OpenClawConfig): AgentBinding[] {
  return Array.isArray(cfg.bindings) ? cfg.bindings : [];
}
```

一个典型的绑定配置如下：

```
// config.json5
{
  "bindings": [
    {
      "agentId": "work-assistant",
      "match": {
        "channel": "slack",
        "accountId": "company-workspace"
      }
    },
    {
      "agentId": "personal",
      "match": {
        "channel": "telegram",
        "peer": { "kind": "dm", "id": "12345" }
      }
    }
  ]
}
```

这表示：来自 Slack 公司工作区的消息交给 `work-assistant` Agent，来自 Telegram 用户 12345 的私聊交给 `personal` Agent。

`listBoundAccountIds()` 函数提取某个通道下所有被绑定的账户 ID：

```
export function listBoundAccountIds(cfg: OpenClawConfig, channelId: string): string[] {
  const normalizedChannel = normalizeBindingChannelId(channelId);
  const ids = new Set<string>();
  for (const binding of listBindings(cfg)) {
    const channel = normalizeBindingChannelId(binding.match?.channel);
    if (channel !== normalizedChannel) continue;
    const accountId = binding.match?.accountId?.trim() ?? "";
    if (!accountId || accountId === "*") continue; // 通配符不算
    ids.add(normalizeAccountId(accountId));
  }
  return Array.from(ids).toSorted((a, b) => a.localeCompare(b));
}
```

`buildChannelAccountBindings()` 构建一个嵌套的 `Map<channel, Map<agentId, accountId[]>>` 结构，方便后续快速查找：

```
export function buildChannelAccountBindings(cfg: OpenClawConfig) {
  const map = new Map<string, Map<string, string[]>>();
  for (const binding of listBindings(cfg)) {
    const channelId = normalizeBindingChannelId(binding.match?.channel);
    if (!channelId) continue;
    const accountId = normalizeAccountId(binding.match?.accountId);
    const agentId = normalizeAgentId(binding.agentId);
    // 构建 channel → agent → accounts 的映射
    const byAgent = map.get(channelId) ?? new Map();
    const list = byAgent.get(agentId) ?? [];
    if (!list.includes(accountId)) list.push(accountId);
    byAgent.set(agentId, list);
    map.set(channelId, byAgent);
  }
  return map;
}
```

13.2.2 路由解析（Resolve Route）

`src/routing/resolve-route.ts` 是路由系统的核心——`resolveAgentRoute()` 函数接收消息的来源信息，返回应该使用的 Agent ID 和会话键。

输入与输出类型

```typescript
export type ResolveAgentRouteInput = {
  cfg: OpenClawConfig;
  channel: string;              // 通道标识（如 "telegram"、"slack"）
  accountId?: string | null;    // 账户 ID
  peer?: RoutePeer | null;      // 消息来源（DM/群组/频道）
  parentPeer?: RoutePeer | null;// 线程的父消息来源
  guildId?: string | null;      // Discord Guild ID
  teamId?: string | null;       // Slack Team ID
};

export type ResolvedAgentRoute = {
  agentId: string;              // 选定的 Agent
  channel: string;
  accountId: string;
  sessionKey: string;           // 生成的会话键
  mainSessionKey: string;       // Agent 的主会话键
  matchedBy: "binding.peer" | "binding.guild" | "binding.account"
    | "binding.channel" | "binding.peer.parent" | "binding.team" | "default";
};
```

`matchedBy` 字段记录了路由匹配的原因，调试时很有用。

匹配优先级

`resolveAgentRoute()` 按照**从具体到宽泛**的优先级尝试匹配：

```
export function resolveAgentRoute(input: ResolveAgentRouteInput): ResolvedAgentRoute {
  // 1. 过滤出匹配通道和账户的绑定
  const bindings = listBindings(input.cfg).filter((binding) => {
    if (!matchesChannel(binding.match, channel)) return false;
    return matchesAccountId(binding.match?.accountId, accountId);
  });

  const choose = (agentId: string, matchedBy: ...) => {
    const resolvedAgentId = pickFirstExistingAgentId(input.cfg, agentId);
    const sessionKey = buildAgentSessionKey({ ... });
    const mainSessionKey = buildAgentMainSessionKey({ ... });
    return { agentId: resolvedAgentId, channel, accountId, sessionKey, mainSessionKey,
  };

  // 优先级 1: 精确对等方匹配（最具体）
  if (peer) {
    const peerMatch = bindings.find((b) => matchesPeer(b.match, peer));
    if (peerMatch) return choose(peerMatch.agentId, "binding.peer");
  }

  // 优先级 2: 线程父对等方匹配（继承父消息的路由）
  if (parentPeer?.id) {
    const parentMatch = bindings.find((b) => matchesPeer(b.match, parentPeer));
    if (parentMatch) return choose(parentMatch.agentId, "binding.peer.parent");
  }

  // 优先级 3: Guild 匹配（Discord 服务器级别）
  if (guildId) {
    const guildMatch = bindings.find((b) => matchesGuild(b.match, guildId));
    if (guildMatch) return choose(guildMatch.agentId, "binding.guild");
  }

  // 优先级 4: Team 匹配（Slack 团队级别）
  if (teamId) {
    const teamMatch = bindings.find((b) => matchesTeam(b.match, teamId));
    if (teamMatch) return choose(teamMatch.agentId, "binding.team");
  }

  // 优先级 5: 账户匹配（排除带有更具体匹配条件的绑定）
  const accountMatch = bindings.find((b) =>
    b.match?.accountId?.trim() !== "*" && !b.match?.peer && !b.match?.guildId
  );
  if (accountMatch) return choose(accountMatch.agentId, "binding.account");
```

```
  // 优先级 6：通道通配符匹配（accountId === "*"）
  const anyAccountMatch = bindings.find((b) =>
    b.match?.accountId?.trim() === "*" && !b.match?.peer && !b.match?.guildId
  );
  if (anyAccountMatch) return choose(anyAccountMatch.agentId, "binding.channel");

  // 优先级 7：默认 Agent
  return choose(resolveDefaultAgentId(input.cfg), "default");
}
```

> **衍生解释**：这种**从具体到宽泛的匹配优先级**在路由系统中很常见。类似于 nginx 的 `location` 匹配规则——精确匹配 > 前缀匹配 > 正则匹配 > 默认。OpenClaw 的路由也遵循同样的原则：peer 绑定 > guild 绑定 > account 绑定 > channel 绑定 > 默认。

13.2.3 会话键生成

路由解析的最终产物是会话键。`buildAgentSessionKey()` 函数根据路由结果生成适当的会话键：

```
export function buildAgentSessionKey(params: {
  agentId: string;
  channel: string;
  accountId?: string | null;
  peer?: RoutePeer | null;
  dmScope?: "main" | "per-peer" | "per-channel-peer" | "per-account-channel-peer";
  identityLinks?: Record<string, string[]>;
}): string {
  const channel = normalizeToken(params.channel) || "unknown";
  const peer = params.peer;
  return buildAgentPeerSessionKey({
    agentId: params.agentId,
    mainKey: DEFAULT_MAIN_KEY,
    channel,
    accountId: params.accountId,
    peerKind: peer?.kind ?? "dm",
    peerId: peer ? normalizeId(peer.id) || "unknown" : null,
    dmScope: params.dmScope,
    identityLinks: params.identityLinks,
  });
}
```

线程会话键

对于 Slack 线程、Discord 帖子等场景， `resolveThreadSessionKeys()` 在基础会话键后追加 `:thread:<threadId>`：

```
export function resolveThreadSessionKeys(params: {
  baseSessionKey: string;
  threadId?: string | null;
  useSuffix?: boolean;
}): { sessionKey: string; parentSessionKey?: string } {
  const threadId = (params.threadId ?? "").trim();
  if (!threadId) return { sessionKey: params.baseSessionKey };
  const sessionKey = params.useSuffix
    ? `${params.baseSessionKey}:thread:${threadId.toLowerCase()}`
    : params.baseSessionKey;
  return { sessionKey, parentSessionKey: params.parentSessionKey };
}
```

13.2.4 键映射规则

用具体示例总结不同消息类型产生的会话键：

直接消息（DM）

```
dmScope=main:
  Telegram 用户 alice → agent:main:main
  Slack 用户 bob     → agent:main:main   (共享！)

dmScope=per-peer:
  Telegram 用户 alice → agent:main:dm:alice
  Slack 用户 bob     → agent:main:dm:bob

dmScope=per-channel-peer:
  Telegram 用户 alice → agent:main:telegram:dm:alice
  Slack 用户 bob     → agent:main:slack:dm:bob
```

群组消息

群组消息**不受 dmScope 影响**，始终包含通道和群组 ID：

```
Telegram 群组 12345    → agent:main:telegram:group:12345
Slack 频道 C789        → agent:main:slack:channel:c789
Discord 频道 #general  → agent:main:discord:channel:general
```

Cron 会话

定时任务使用特殊的键格式：

```
每日报告任务        → agent:main:cron:daily-report
单次运行           → agent:main:cron:daily-report:run:<uuid>
```

线程会话

线程在基础键上追加线程标识：

```
Slack 频道 C789 中的线程 T456:
  基础键:  agent:main:slack:channel:c789
  线程键:  agent:main:slack:channel:c789:thread:t456
```

本节小结

会话路由是 OpenClaw 消息处理流水线中的关键一环：

1. **绑定系统**：通过配置文件将通道/账户/对等方映射到特定的 Agent
2. **优先级匹配**：从具体（peer）到宽泛（default）的 7 级匹配优先级
3. **灵活的键生成**：根据 `dmScope` 、 `identityLinks` 和消息类型生成唯一的会话键
4. **线程继承**：线程消息继承父消息的路由绑定，保证同一线程的所有消息路由到同一个 Agent

路由系统的设计使得 OpenClaw 能够在一个 Gateway 实例中同时服务多个 Agent、多个通道、多种消息类型，而每条消息都能被准确地路由到正确的位置。